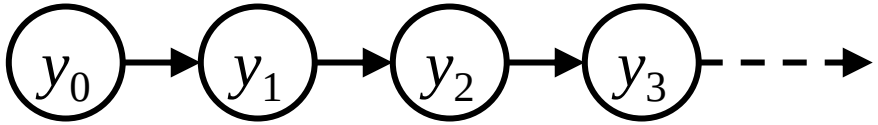
50.007 Machine Learning

Lecture 18

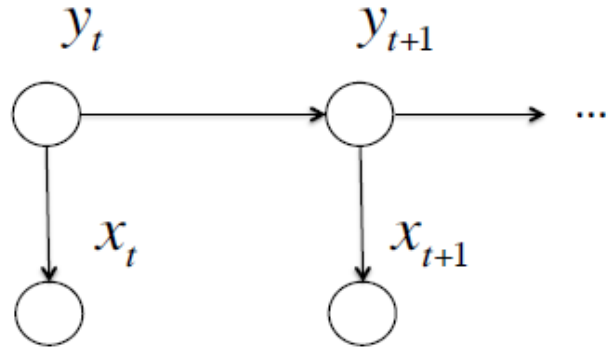
Markov Decision Process

Markov Chains, HMM, MDP

- Markov chains are transition over states

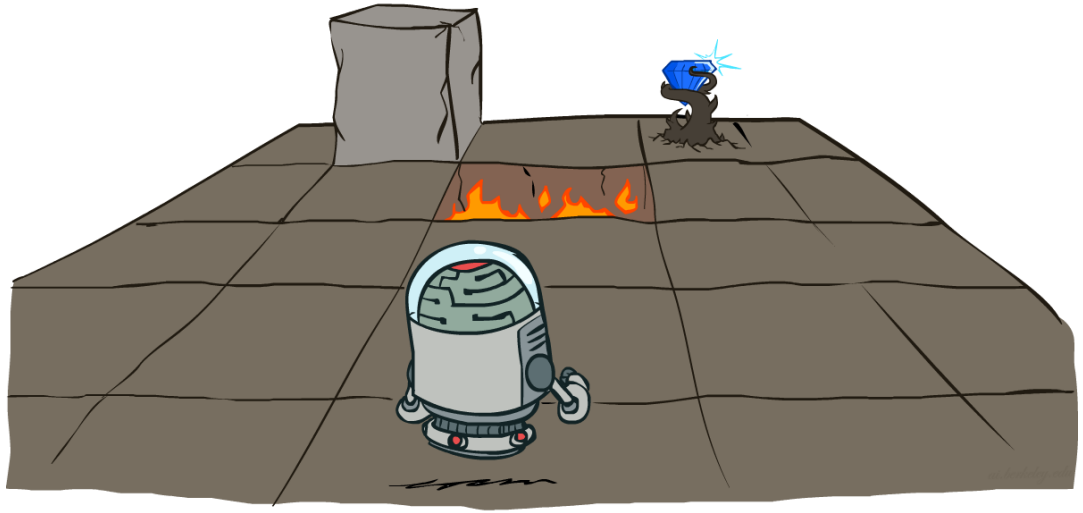


- Hidden Markov models (HMM)
  - Underlying Markov chain over states y ∈ Y
  - You observe outputs (effects) at each time step
  - E.g. Robot localization problem



- Markov Decision Process (MDP)
  - States are directly observable
  - Accounting for actions taken by the robot

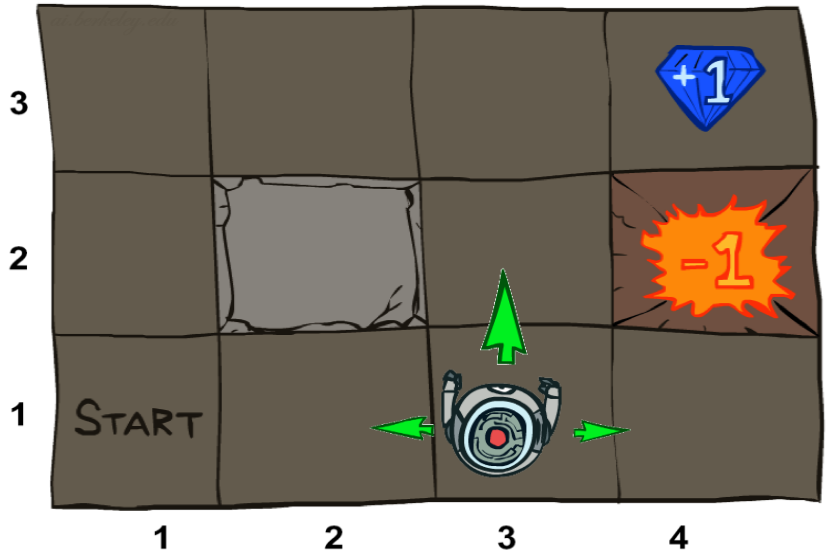
Learn How to Act



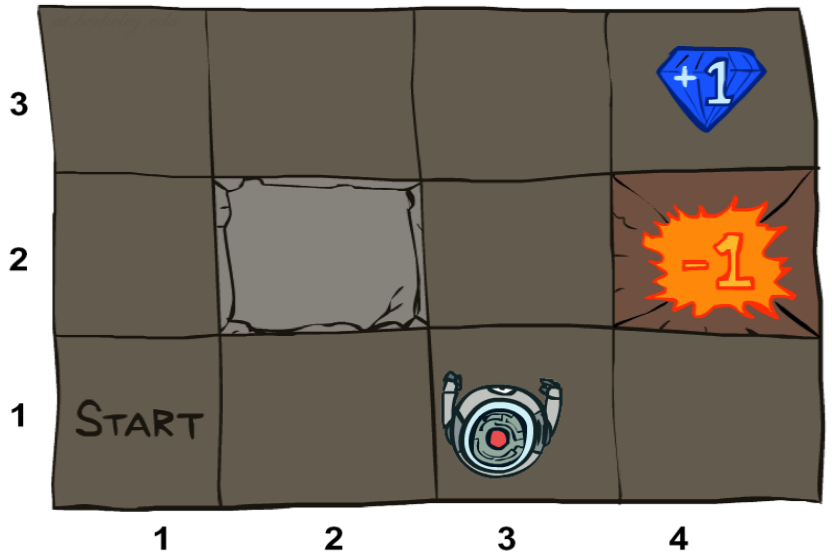
How do we teach a robot to act optimally
in a complex environment?

Example: Grid World

- A maze-like problem
  - The agent lives in a grid
  - Walls block the agent's path
- Noisy movement: actions do not always go as planned
  - E.g. 80% of the time, the action North takes the agent North (if there is no wall there)
  - 10% of the time, North takes the agent West; 10% East
  - If there is a wall in the direction the agent would have been taken, the agent stays put
- The agent receives rewards each time step
  - Small "living" reward each step (can be negative)
  - Terminal rewards come at the end (good or bad)
- Goal: maximize sum of rewards



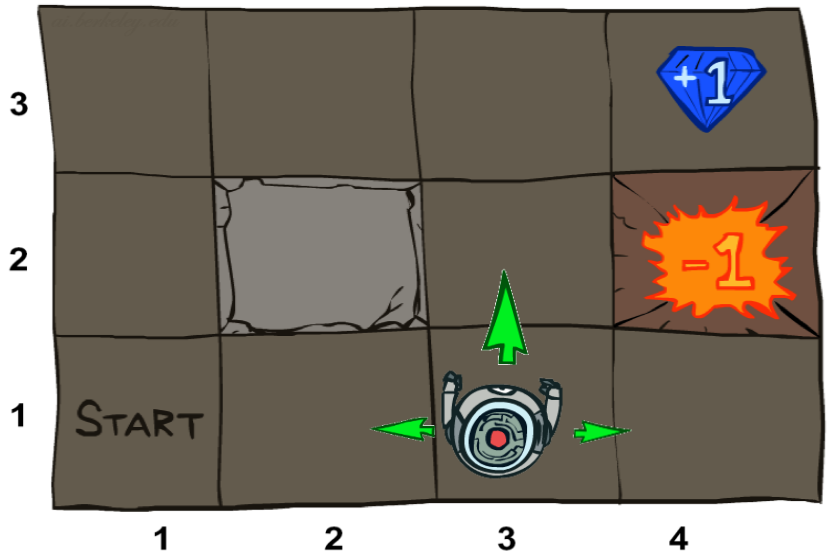
Grid World States



The robot can be at one block at any given time.

There is a set of states S

Grid World Actions

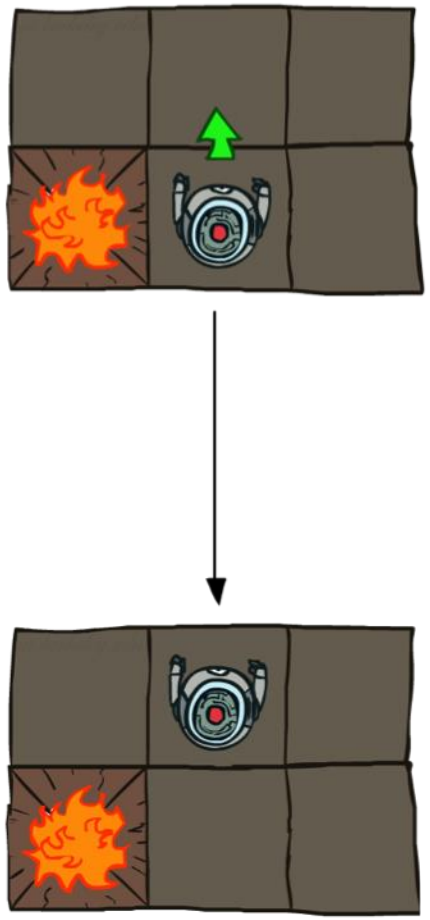


The robot can take an action from a set of predefined possible actions at each state.

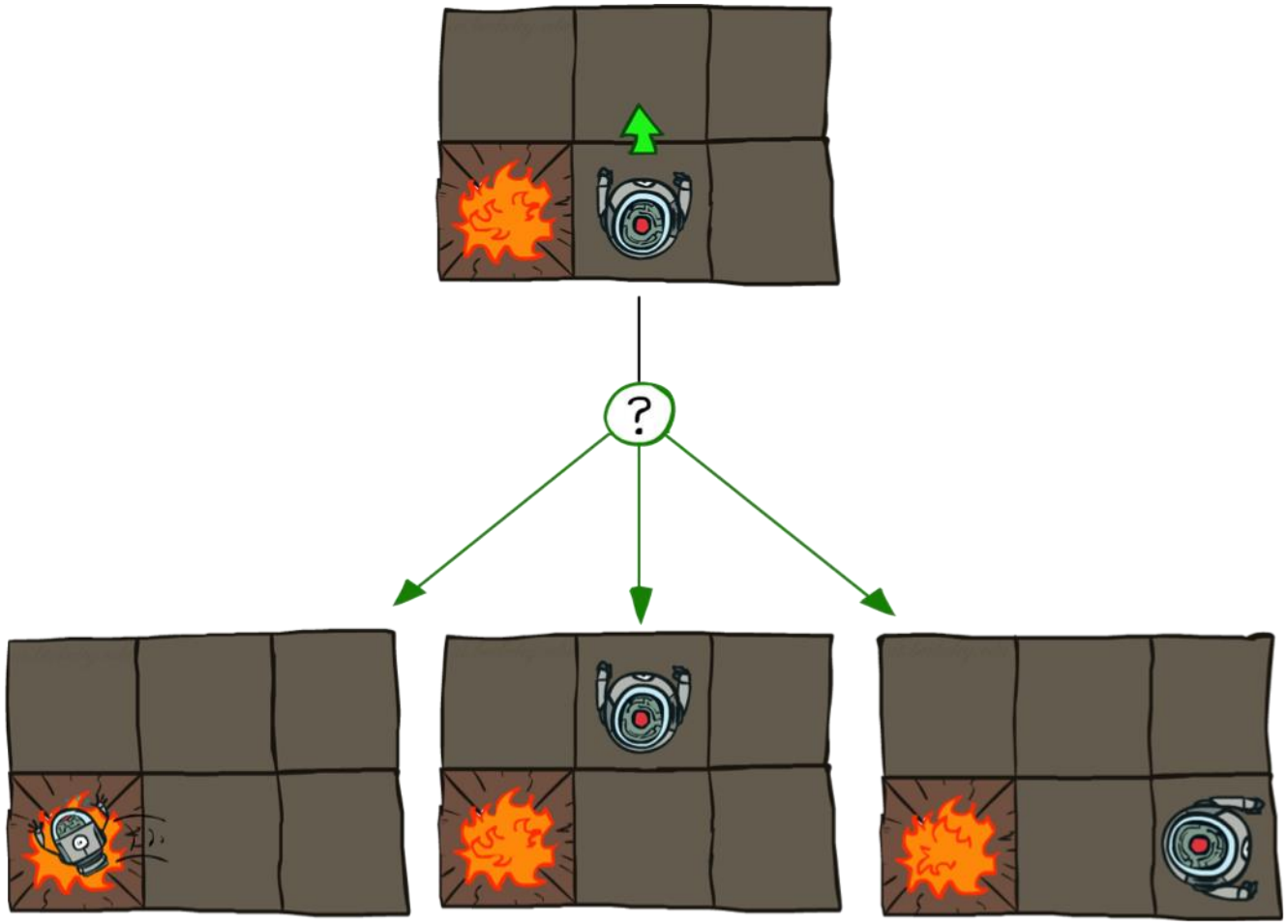
There is a set of actions A

Grid World Actions

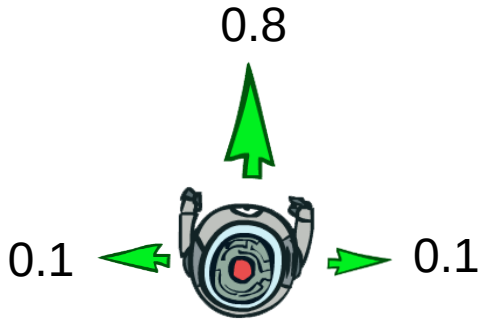
Deterministic Grid World



Stochastic Grid World



Grid World Transition



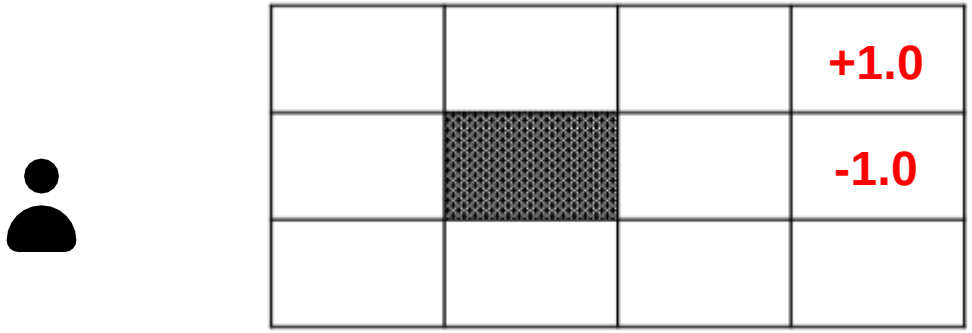
If the robot moves towards a particular direction, there is a 0.8 chance that it would reach the block in front, and there is a 0.1 chance to reach a state to its left (right).

A transition probability function

$$T(s, a, s') = p(s'|s, a)$$

The R of s' given in state s for an action.

Grid World Rewards



Each block is associated with a small “living” reward (can be negative).
There is a terminal state reward which can be good or bad (the two blocks at the upper-right corner).

Grid World Rewards

-0.6	+1.2	+0.1	+1.0
-0.1		-0.1	-1.0
+0.9	-0.7	-2.0	+1.3

Each block is associated with a small “living” reward (can be negative). There is a terminal state reward which can be good or bad (the two blocks at the upper-right corner).

The reward is $R(s)$ for each state.

In general it can be defined as $R(s, a, s')$

action

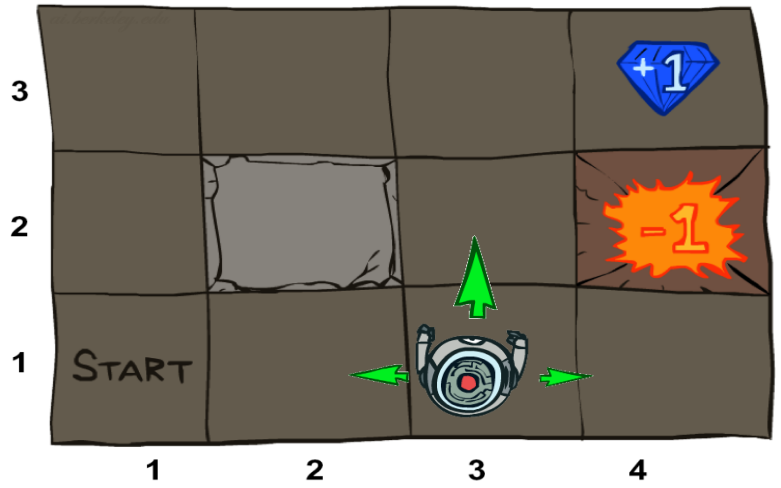
old state

new state

Markov Decision Process (MDP)

An MDP is defined by:

- A set of states $s \in S$
- A set of actions $a \in A$
- A transition function T(s, a, s')
  - Probability that a from s leads to s', i.e., P(s'| s, a)
  - Also called the model or the dynamics
- A reward function R(s, a, s')
  - Sometimes just R(s) or R(s')
- A start state
- Maybe a terminal state



-0.6	+1.2	+0.1	+1.0
-0.1		-0.1	-1.0
+0.9	-0.7	-2.0	+1.3

What is Markov about MDPs?

- "Markov" generally means that given the present state, the future and the past are independent

- For Markov decision processes, "Markov" means action outcomes depend only on the current state

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \ldots S_0 = s_0)$$

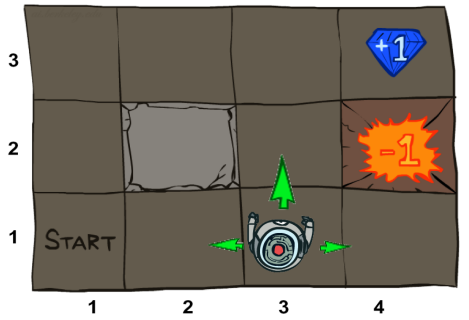
$$=$$

$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$

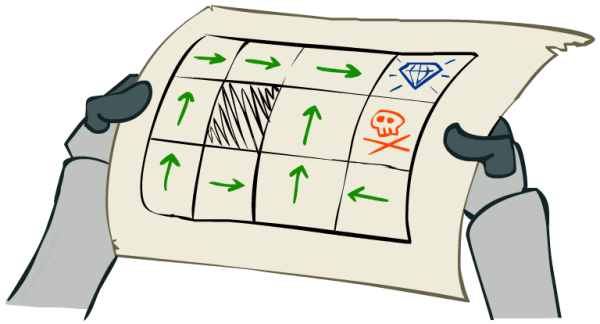
Andrey Markov
(1856-1922)

Markov Decision Process (MDP)

- An MDP is defined by:
  - A set of states $s \in S$
  - A set of actions $a \in A$
  - A transition function $T(s, a, s')$
  - A reward function $R(s, a, s')$
  - A start state
  - Maybe a terminal state
- Goal: maximize sum of rewards



-0.6	+1.2	+0.1	+1.0
-0.1		-0.1	-1.0
+0.9	-0.7	-2.0	+1.3



We want an optimal policy π^*, that gives the best action for each state such that we maximize the overall sum of rewards.

Optimal policy $\pi^*: S \rightarrow A$

Grid World
Utility (Long Term Reward)

-0.6	+1.2	+0.1	+1.0
0.1		-0.1	-1.0
+0.9	-0.7	-2.0	+1.3

$$U([s_0, s_1, s_2, \dots]) = R(s_0) + R(s_1) + R(s_2) + \cdots = \sum_{t=0}^{\infty} R(s_t)$$

Will this be a good way of defining long term reward?

Grid World
Utility (Long Term Reward)

-0.6	→	←	+1.0
0.1		-0.1	-1.0
+0.9	-0.7	-2.0	+1.3

$$U([s_0, s_1, s_2, \dots]) = R(s_0) + R(s_1) + R(s_2) + \cdots = \sum_{t=0}^{\infty} R(s_t)$$

Will this be a good way of defining long term reward?

Grid World
Utility (Long Term Reward)

- It's reasonable to maximize the sum of rewards
- It's also reasonable to prefer rewards now to rewards later
- **One solution:** values of rewards decay exponentially

1

Worth
Now

γ

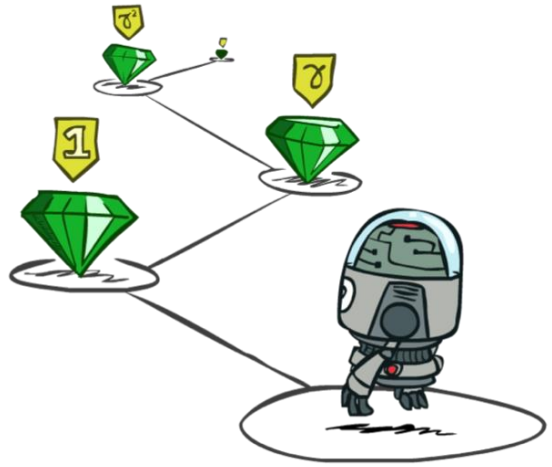
Worth
Next Step

γ^2

Worth In
Two Steps

Grid World
Utility (Long Term Reward)

-0.6	+1.2	+0.1	+1.0
0.1		-0.1	-1.0
+0.9	-0.7	-2.0	+1.3

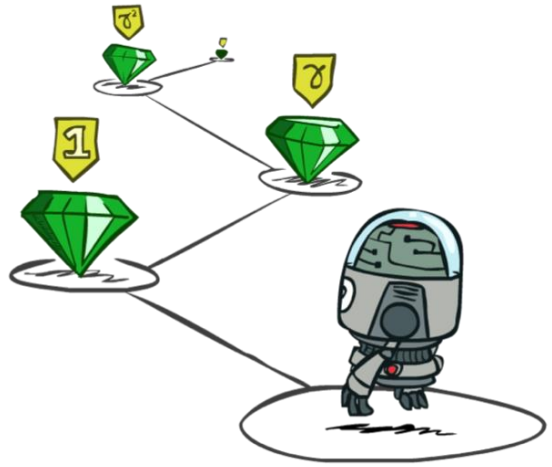


$$U([s_0, s_1, s_2, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots = \sum_{t=0}^{\infty} \gamma^t R(s_t)$$

discount factor

Grid World
Utility (Long Term Reward)

-0.6	+1.2	+0.1	+1.0
0.1		-0.1	-1.0
+0.9	-0.7	-2.0	+1.3



$$U([s_0, s_1, s_2, \dots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots = \sum_{t=0}^{\infty} \gamma^t R(s_t)$$

It will converge to this term!

$$\frac{R_{min}}{1-\gamma} = \sum_{t=0}^{\infty} \gamma^t R_{min} \leq U([s_0, s_1, s_2, \dots]) \leq \sum_{t=0}^{\infty} \gamma^t R_{max} = \frac{R_{max}}{1-\gamma}$$

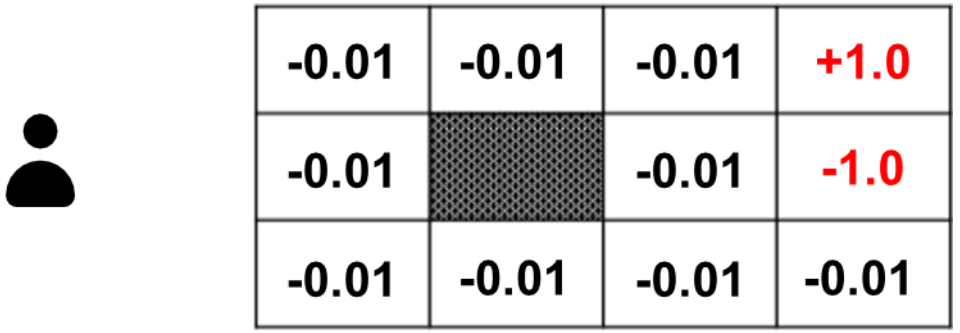
Lower Bound

Smallest Reward

Largest Reward

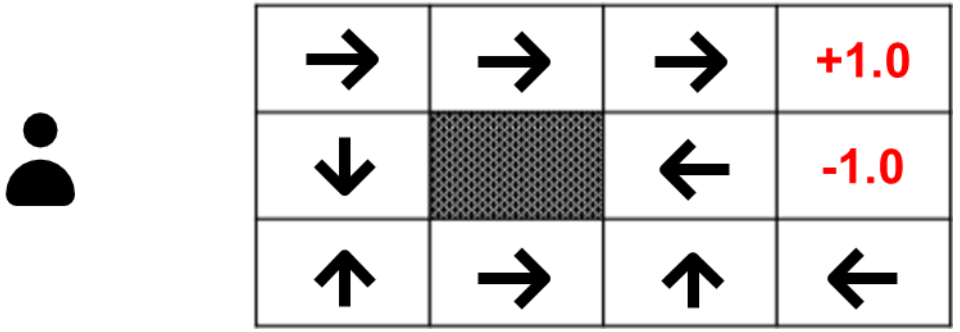
Upper Bound

Grid World
Learning the Policy



How to learn the policy?

R(s) = -0.01

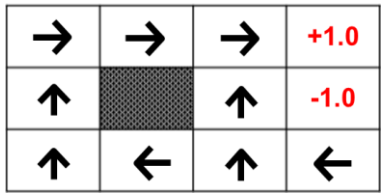


Policy, Value, Q-Value

$\pi(s)$

a particular *policy* that specifies

the action we should take in state s.

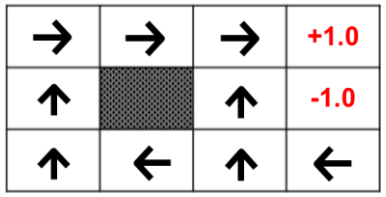


Policy, Value, Q-Value

$\pi(s)$

a particular *policy* that specifies

the action we should take in state s.

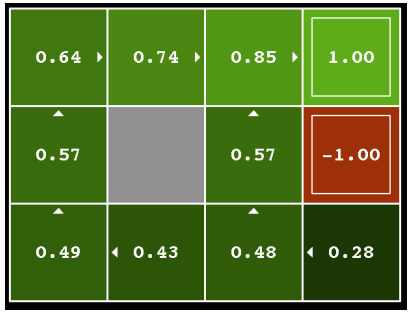


$V^{\pi}(s)$

The *value* of state s under policy π.

It is the expected long-term reward of starting in state s

and act based on policy π thereafter.



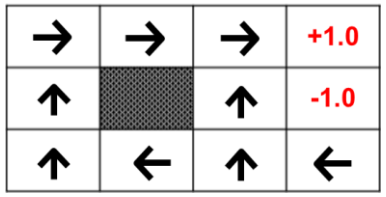
From current state, what is the average long-term reward based on policy π.

Policy, Value, Q-Value

$\pi(s)$

a particular *policy* that specifies

the action we should take in state s.

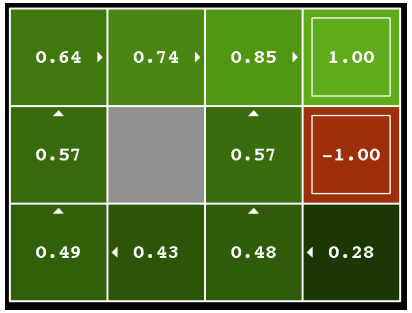


$V^{\pi}(s)$

The *value* of state s under policy π.

It is the expected long-term reward of starting in state s

and act based on policy π thereafter.

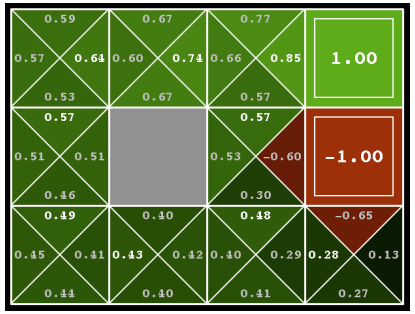


$Q^{\pi}(s, a)$

The *Q-value* of state s and action a under policy π.

It is the expected long-term reward of starting in state s,

taking action a and acting based on policy π thereafter.

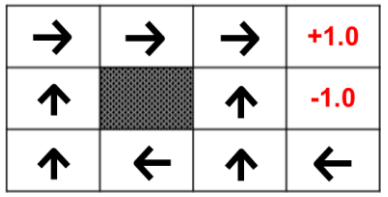


Optimal Policy, Value, Q-Value

taking the best action at each state & getting the max. reward.

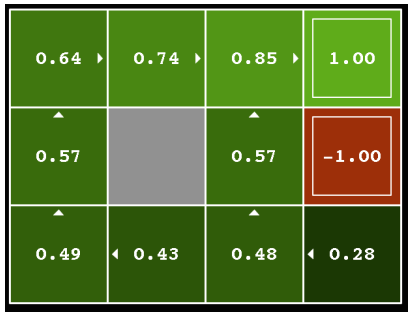
$\pi^*(s)$

The *optimal policy* $\pi^*(s)$ specifies the optimal action we should take in state s.



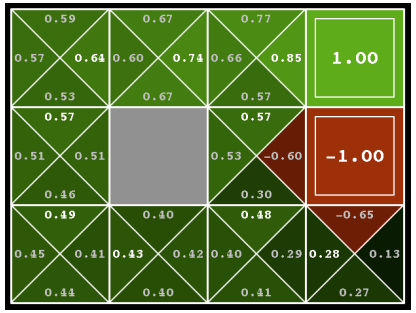
$V^*(s)$

The *value* of state s under the optimal policy π^*



$Q^*(s, a)$

The *Q-value* of state s and action a under the optimal policy π^*



Optimal Value

$V^*(s) = ??$ $\max_a Q^*(s, a)$

$V^*(s)$

The *value* of state s under the optimal policy π^*

$Q^*(s, a)$

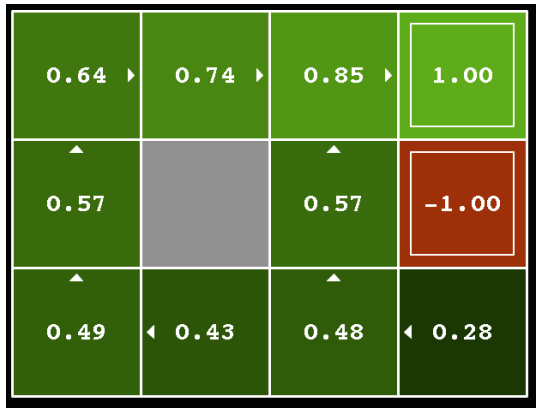
The Q-*value* of state s

and action a under the optimal policy π^*

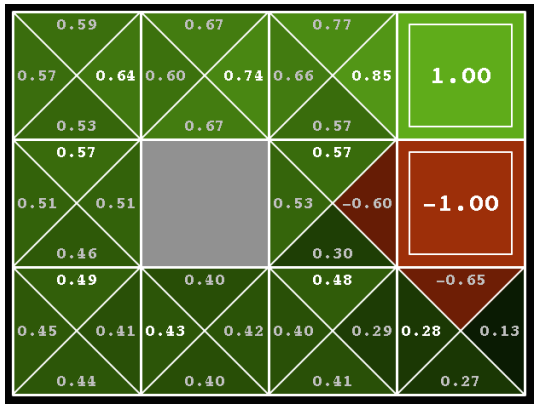
Optimal Value

By taking the max. value at each grid = $V^*(s)$

$$V^*(s) = \max_a Q^*(s,a)$$



$= \max_a ($



$)$

$V^*(s)$

The *value* of state s under the optimal policy π^*

$Q^*(s,a)$

The Q-*value* of state s

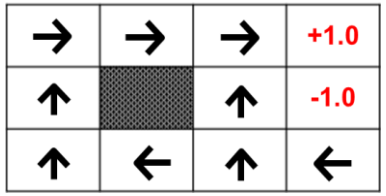
and action a under the optimal policy π^*

Optimal Value

$$V^*(s) = \max_a Q^*(s,a) = Q^*(s, \pi^*(s))$$

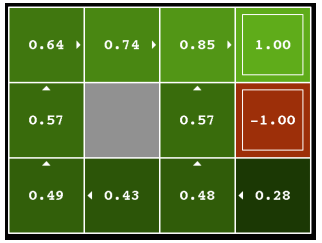
$\pi^*(s)$

The *optimal policy* $\pi^*(s)$ specifies the optimal action we should take in state s.



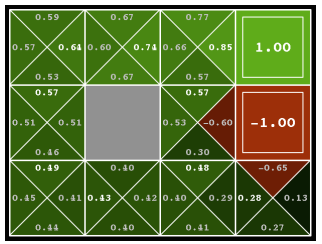
$V^*(s)$

The *value* of state s under the optimal policy π^*



$Q^*(s,a)$

The *Q-value* of state s and action a under the optimal policy π^*

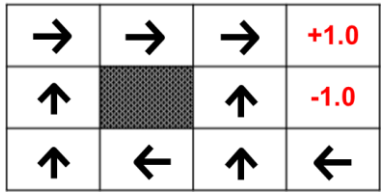


Optimal Q-Value

$$Q^*(s,a) = \; ??$$

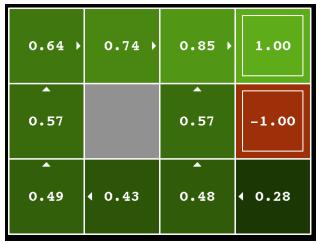
$\pi^*(s)$

The *optimal policy* $\pi^*(s)$ specifies the optimal action we should take in state s.



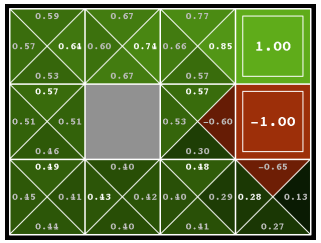
$V^*(s)$

The *value* of state s under the optimal policy π^*



$Q^*(s,a)$

The Q-*value* of state s and action a under the optimal policy π^*



Optimal Q-Value

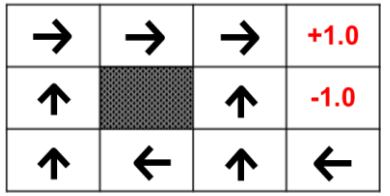
Expected long-term reward defined by $Q^*(s,a)$

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

- $T(s,a,s')$: probability of next state
- $R(s,a,s')$: immediate reward
- $V^*(s')$: long-term reward

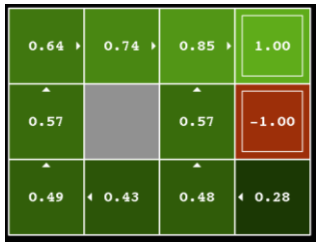
$\pi^*(s)$

The *optimal policy* $\pi^*(s)$ specifies the optimal action we should take in state s.



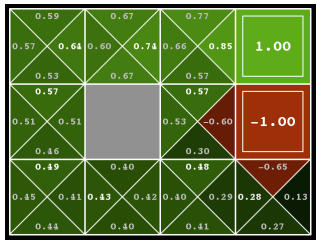
$V^*(s)$

The *value* of state s under the optimal policy π^*



$Q^*(s,a)$

The *Q-value* of state s and action a under the optimal policy π^*



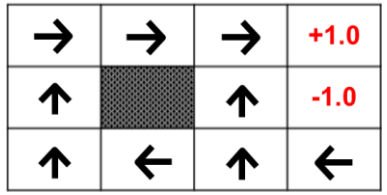
Optimal Value

$$V^*(s) = \max_a Q^*(s,a)$$

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

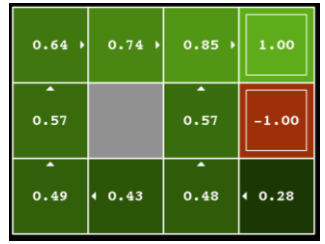
$\pi^*(s)$

The *optimal policy* $\pi^*(s)$ specifies the optimal action we should take in state s.



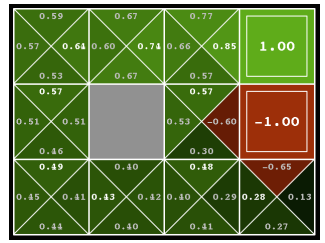
$V^*(s)$

The *value* of state s under the optimal policy π^*



$Q^*(s,a)$

The *Q-value* of state s and action a under the optimal policy π^*



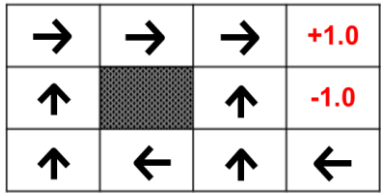
Optimal Value

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Recursive equation

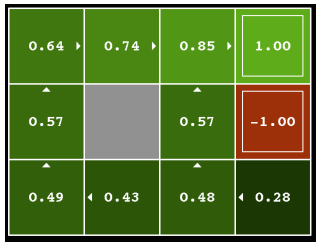
$\pi^*(s)$

The *optimal policy* $\pi^*(s)$ specifies the optimal action we should take in state s.



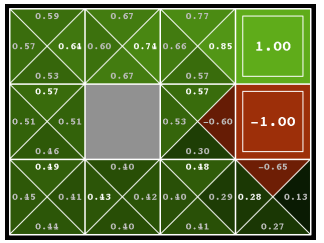
$V^*(s)$

The *value* of state s under the optimal policy π^*



$Q^*(s,a)$

The *Q-value* of state s and action a under the optimal policy π^*



Optimal Value

$$V^*(s) = \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V^*(s')]$$

Updating Value

$$V^*(s) = \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V^*(s')]$$

$$V^*(s) \leftarrow \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V^*(s')]$$

Recursive update

Value Iteration

1. Start with $V_0^*(s) = 0$, for all $s \in S$

2. Given V_i^*, calculate the values for all states $s \in S$

$$V_{i+1}^*(s) \leftarrow \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V_i^*(s')]$$

3. Repeat the above until convergence

Value Iteration

1. Start with $V_0^*(s) = 0$, for all $s \in S$

2. Given V_i^*, calculate the values for all states $s \in S$

$$V_{i+1}^*(s) \leftarrow \max_a \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma V_i^*(s')]$$

3. Repeat the above until convergence

There is a guarantee that this process will converge

Question

How do we recover the optimal policy based on the values?

Recall: Optimal Value, Q-Value, Policy

$$V^*(s) = \max_a Q^*(s,a) = Q^*(s, \pi^*(s))$$

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Also, known as Bellman Equation

Recall: Optimal Value, Q-Value, Policy

$$V^*(s) = \max_a Q^*(s,a) = Q^*(s, \pi^*(s))$$

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Optimal Policy

$$\pi^*(s) = ??$$

$\pi^*(s)$

The *optimal policy* $\pi^*(s)$ specifies the optimal action we should take in state s.

$Q^*(s, a)$

The *Q-value* of state s and action a under the optimal policy π^*

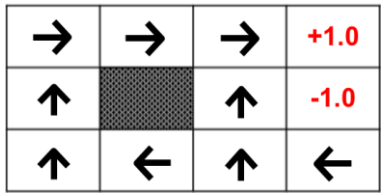
Optimal Policy

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$

$\pi^*(s)$

The *optimal policy* $\pi^*(s)$ specifies

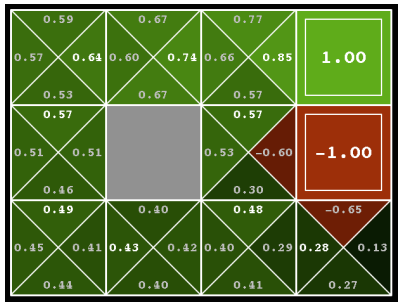
the optimal action we should take in state s.



$Q^*(s,a)$

The *Q-value* of state s

and action a under the optimal policy π^*



Learning the Optimal Policy

Step 1
Run Value Iteration Algorithm

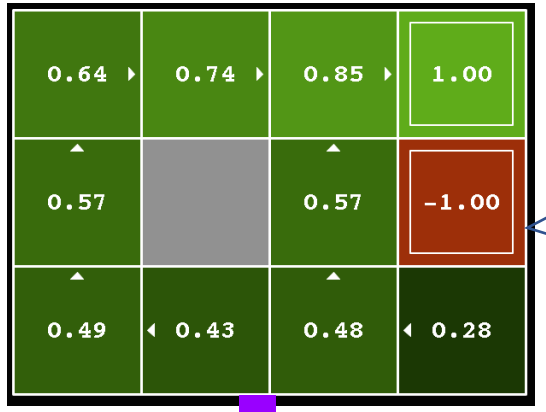
$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Step 2
Calculate the Q values

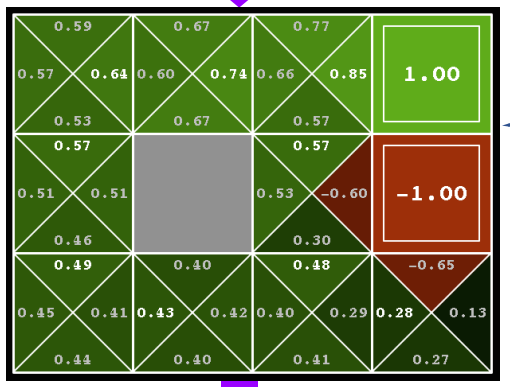
$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Step 3
Find the optimal action for each state

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$

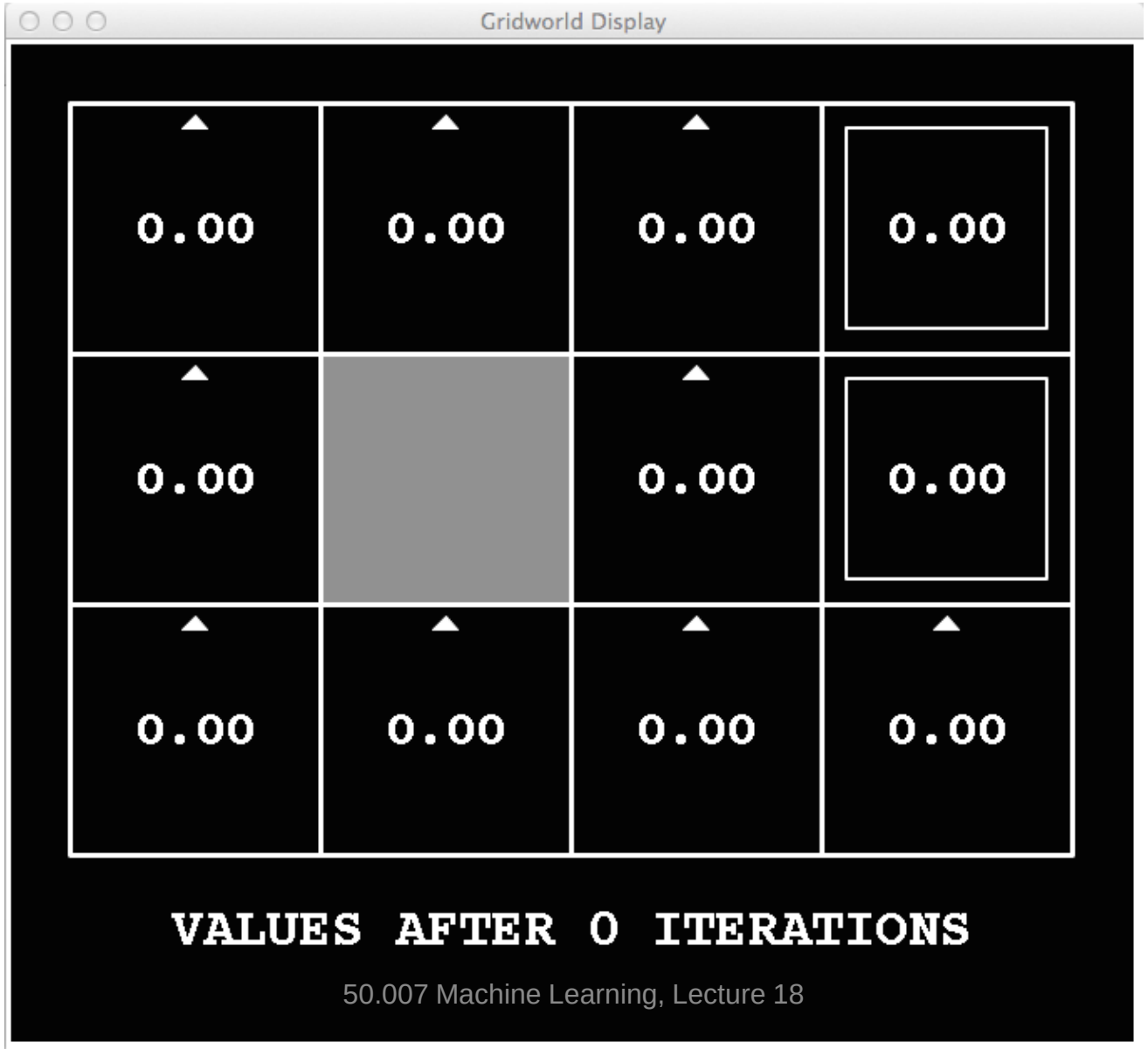


Remember we only kept track of the optimal values and not the actions (i.e. little arrows are not known).



Extra step needed to extract the policy.

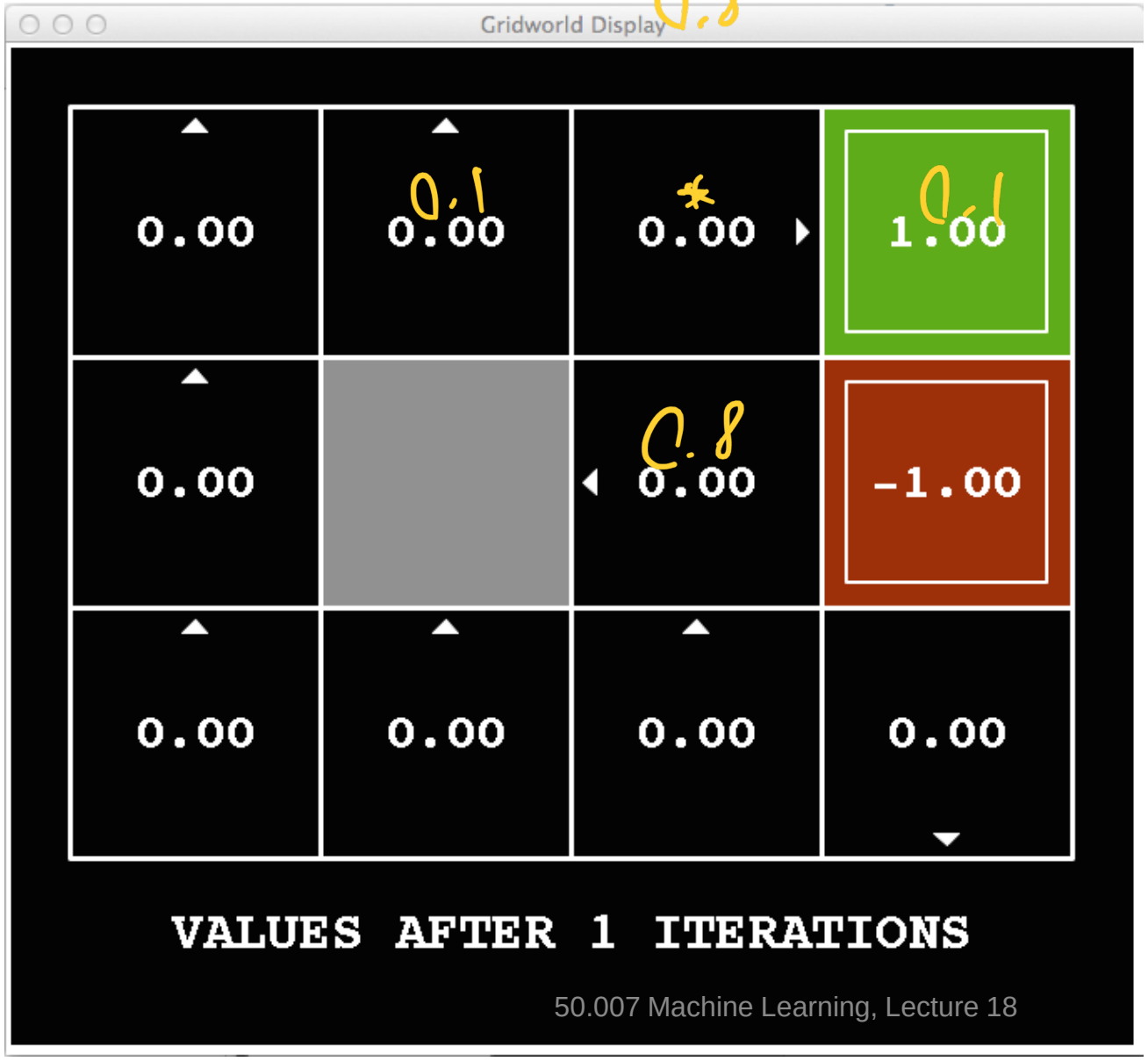
Value Iteration Example: i=0



Noise = 0.2
Discount = 0.9
Living reward = 0

Noise = 0.2
(i.e. move successful with p=0.8;
deviation to left/right both with p=0.1)

i=1



Implementation of terminal state '*e*' with reward '*r*':

- 1 action: '*x*' (exit)
- $T(e, x) = 1$
- $R(e, x) = r$

→ In k=1 only action from terminal states results in change in *V*-values

Noise = 0.2
Discount = 0.9
Living reward = 0

i=2

$$V^*_{i+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*_i(s')]$$

let s: robot in (3,3) then V(s):

$a = north$

0.8·[0 + 0.9·0] + 0.1·[0 + 0.9·0] +
0.1·[0 + 0.9·1] = 0.09

$a = west$

0.8·0.9·0 + 0.1·0.9·0 + 0.1·0.9·0 = 0

$a = south$

0.8·0.9·0 + 0.1·0.9·0 + 0.1·0.9·1 = 0.09

$a = east$

0.8·0.9·1 + 0.1·0.9·0 + 0.1·0.9·0 = 0.72

0.8[0 + 0.9·1] + 0.1[0 + 0.9·0]
+ 0.1[0 + 0.9·0]

= 0.8·0.9 + 0 + 0

= 0.72

Noise = 0.2
Discount = 0.9
Living reward = 0

i=3

γ: 0.8·0.78·0.9 + 0.1·0.52·0.9 + 0.1·1·0.9 = 0.6984

S: 0.8·0.43·0.9 + 0.1·1·0.9 + 0.1·0.52·0.9 = 0.4464

E: 0.8·1·0.9 + 0.1·0.78·0.9 + 0.1·0.43·0.9 = 0.83

$V_{i+1}^*(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_i^*(s')]$

let s: robot in (3,3)
recall $V_2(s) = 0.72$; *now* $V_3(s)$:

$a = north$
0.8·[0 + 0.9·0.72] + 0.1·[0 + 0.9·0] + 0.1·[0 + 0.9·1] = 0.6084

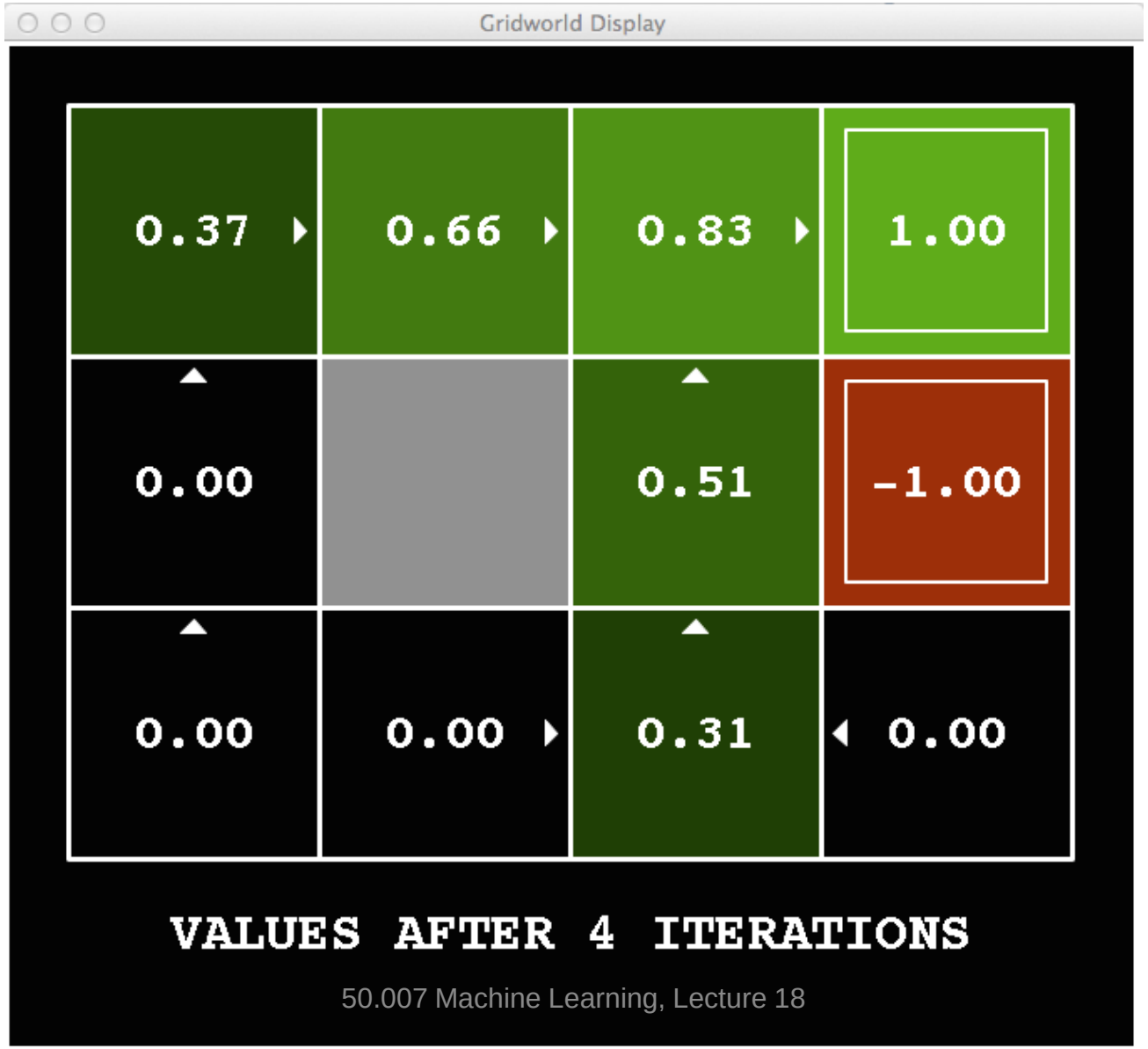
$a = west$
0.8·0.9·0 + 0.1·0.9·0.72 + 0.1·0.9·0 = 0.0648

$a = south$
0.8·0.9·0 + 0.1·0.9·0 + 0.1·0.9·1 = 0.09

$a = east$
0.8·0.9·1 + 0.1·0.9·0.72 + 0.1·0.9·0 = 0.7848

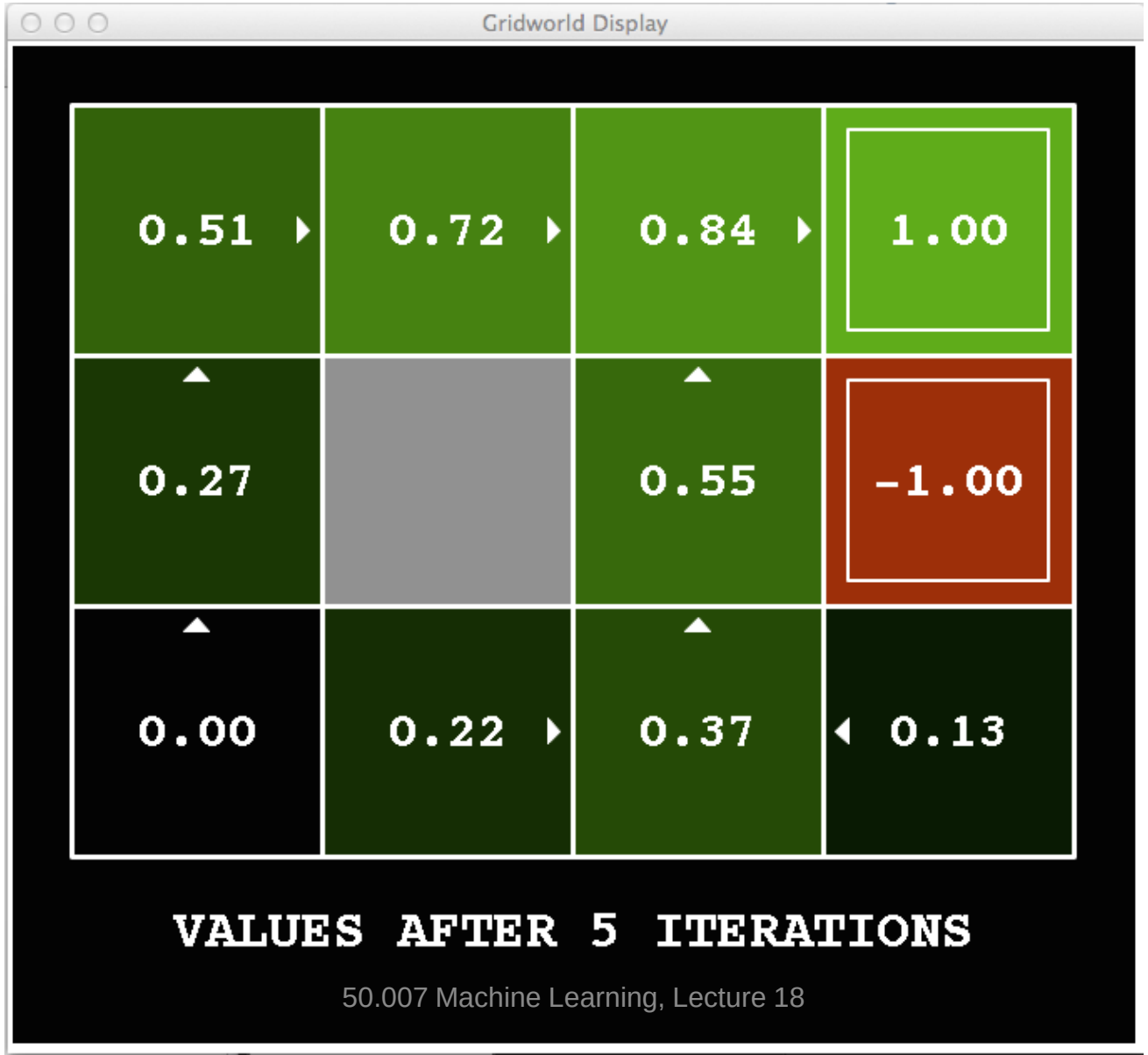
Noise = 0.2
Discount = 0.9
Living reward = 0

i=4



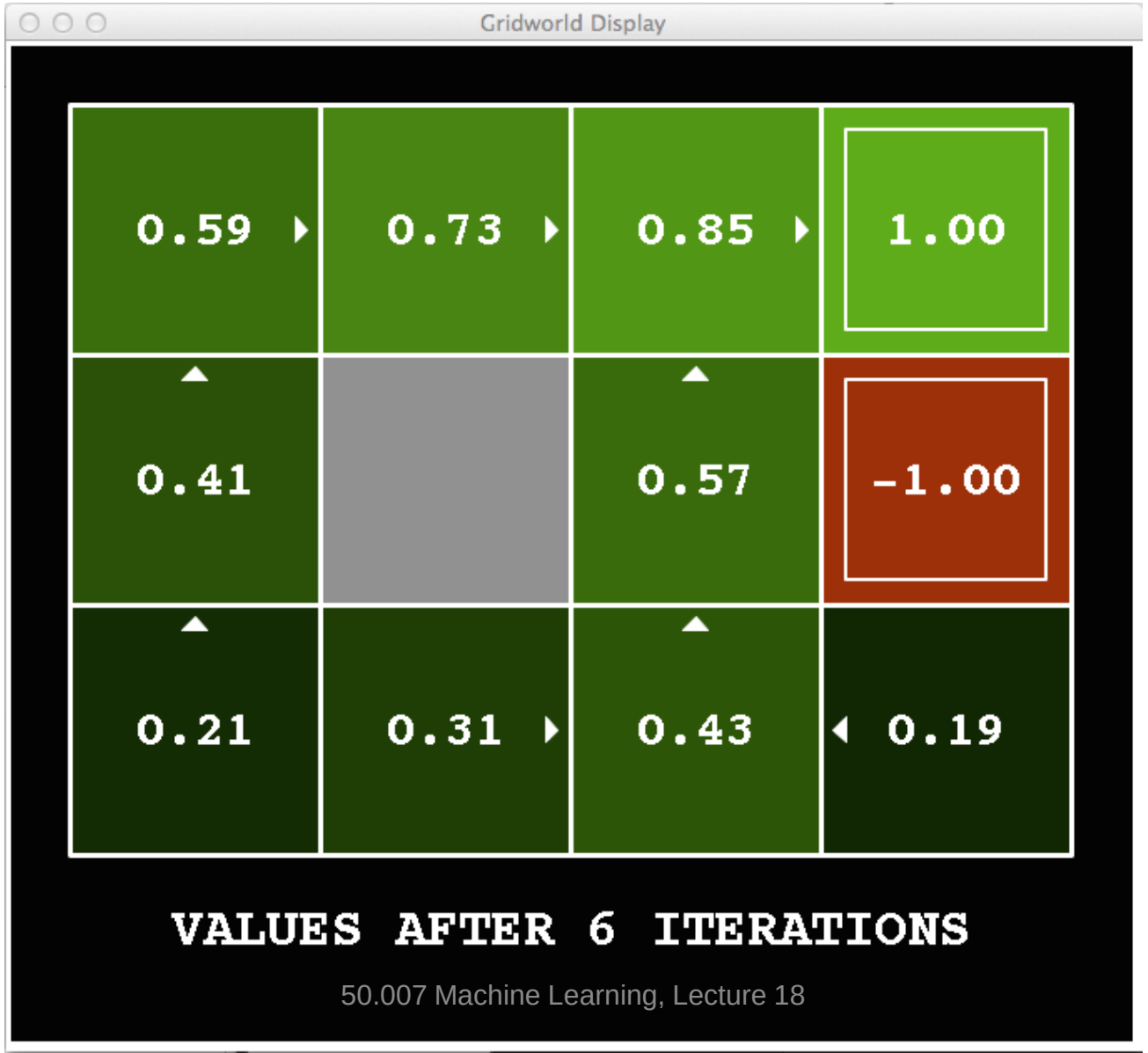
Noise = 0.2
Discount = 0.9
Living reward = 0

i=5



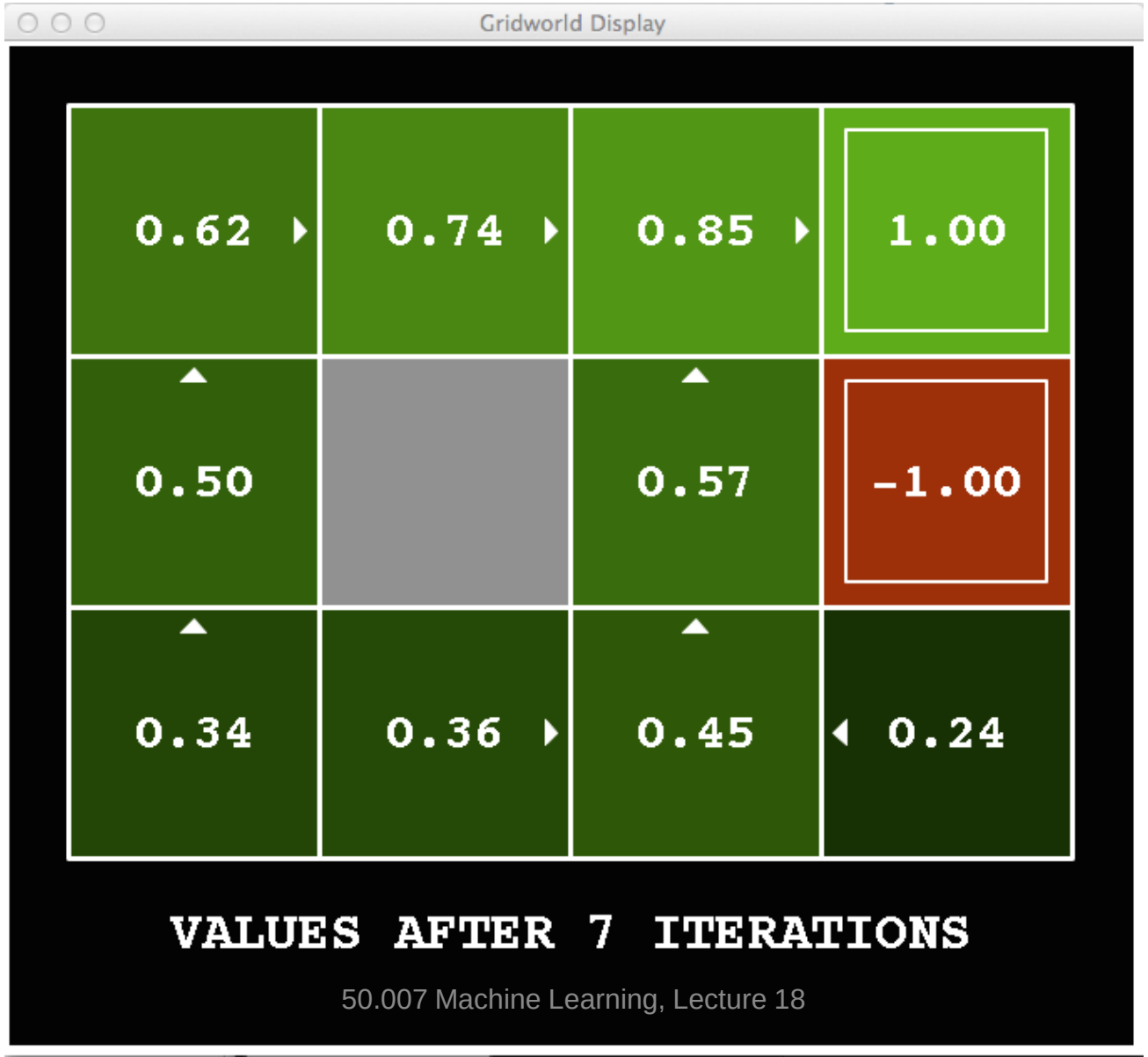
Noise = 0.2
Discount = 0.9
Living reward = 0

i=6



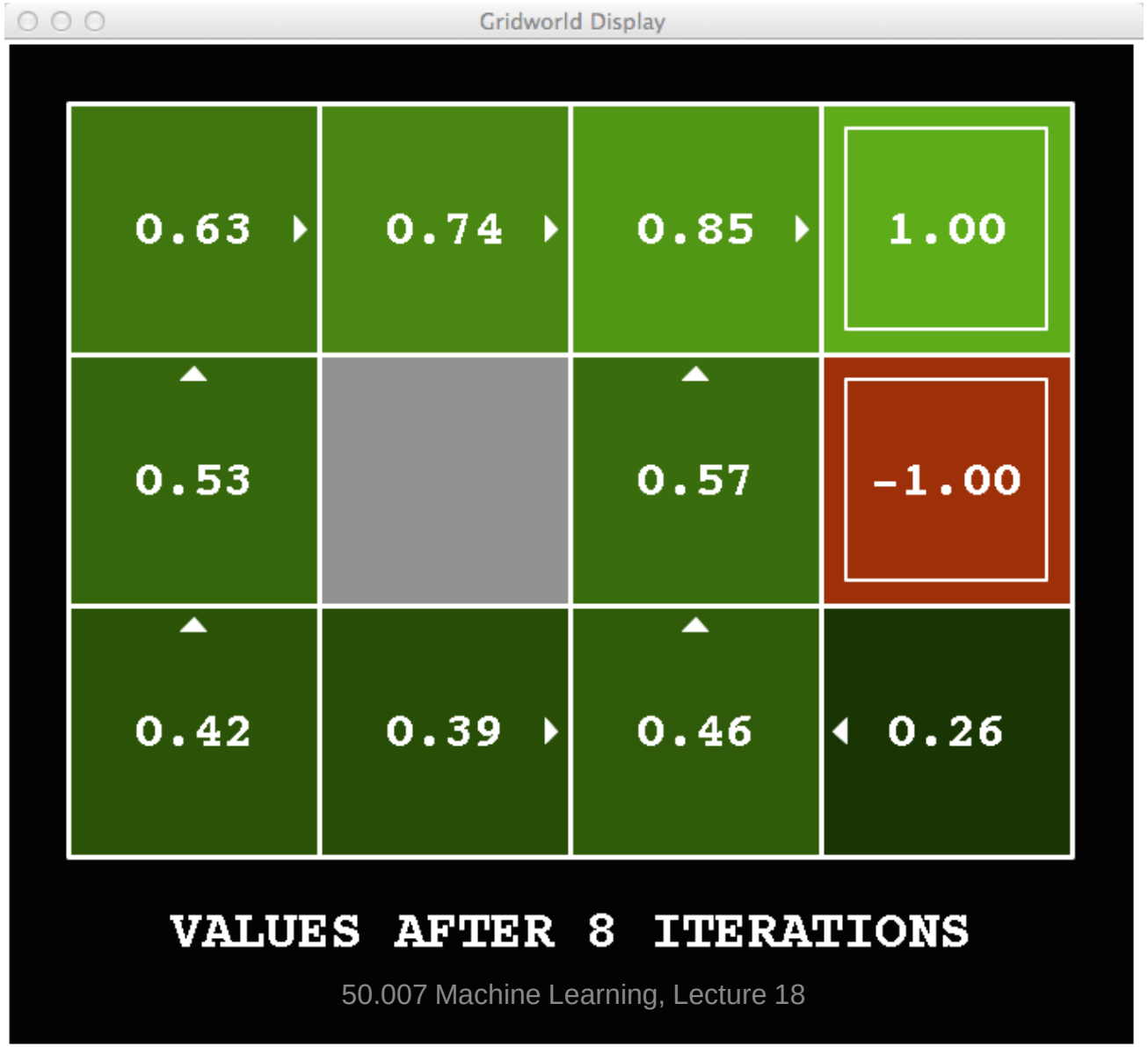
Noise = 0.2
Discount = 0.9
Living reward = 0

i=7



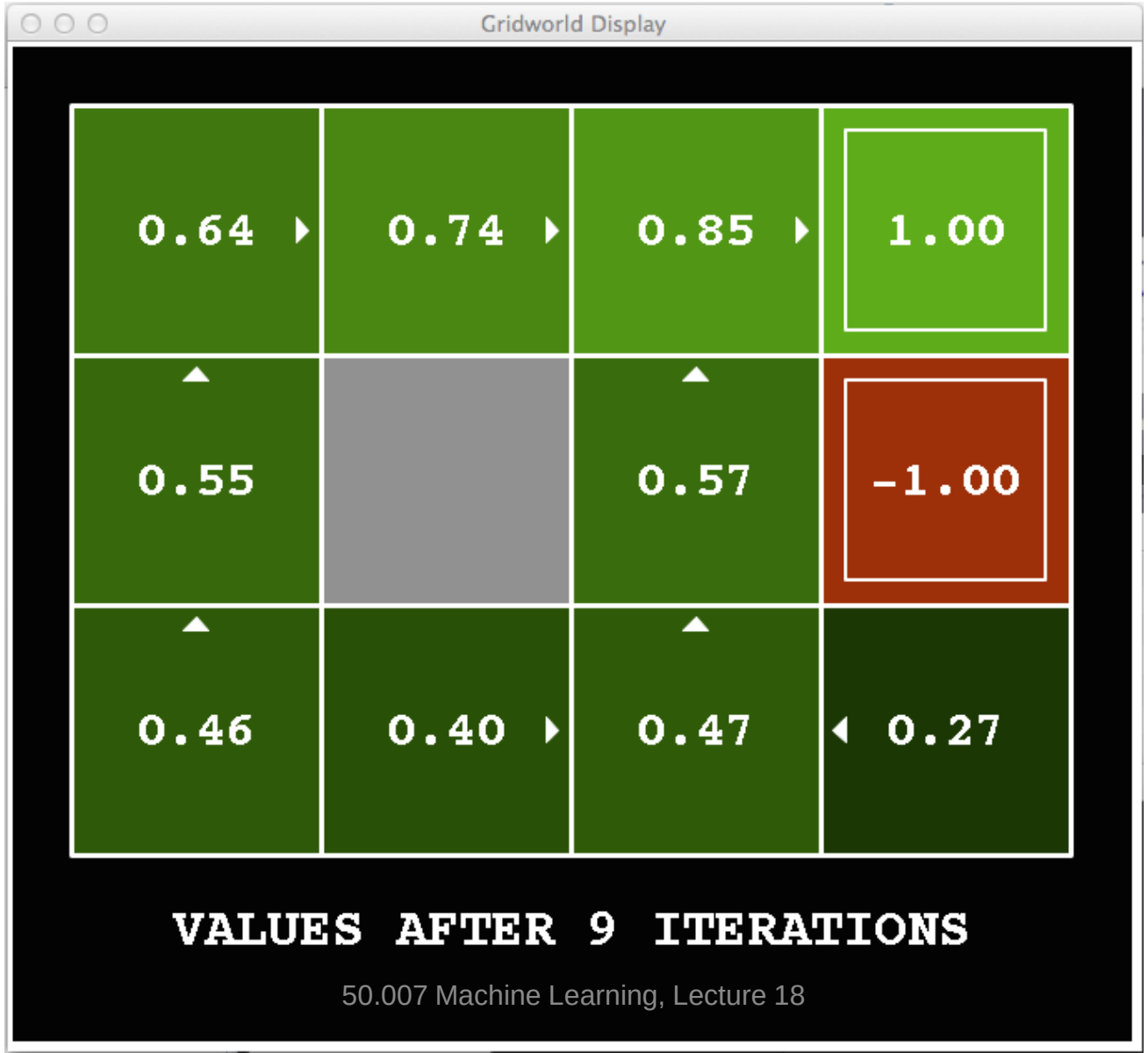
Noise = 0.2
Discount = 0.9
Living reward = 0

i=8



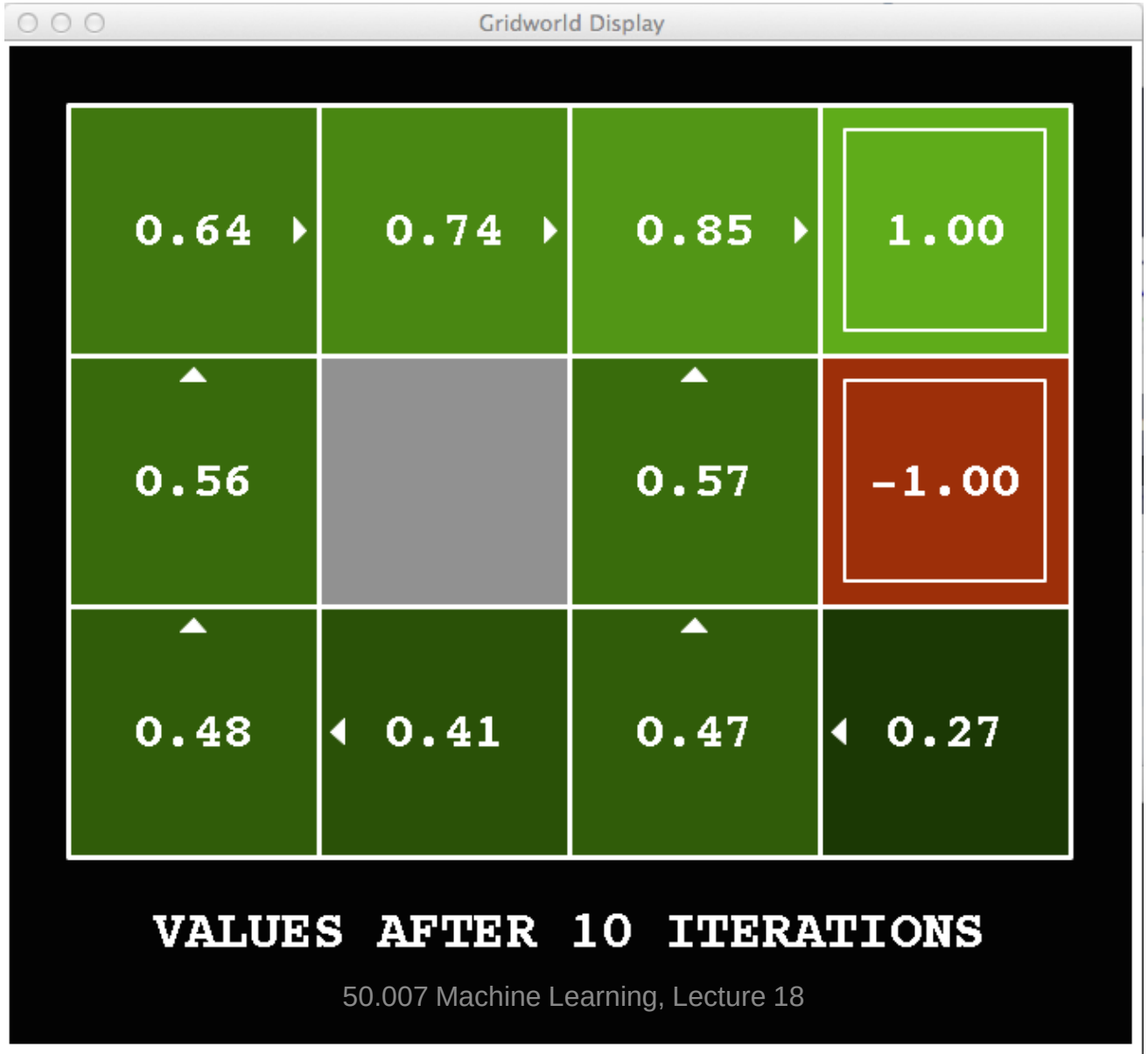
Noise = 0.2
Discount = 0.9
Living reward = 0

i=9



Noise = 0.2
Discount = 0.9
Living reward = 0

i=10

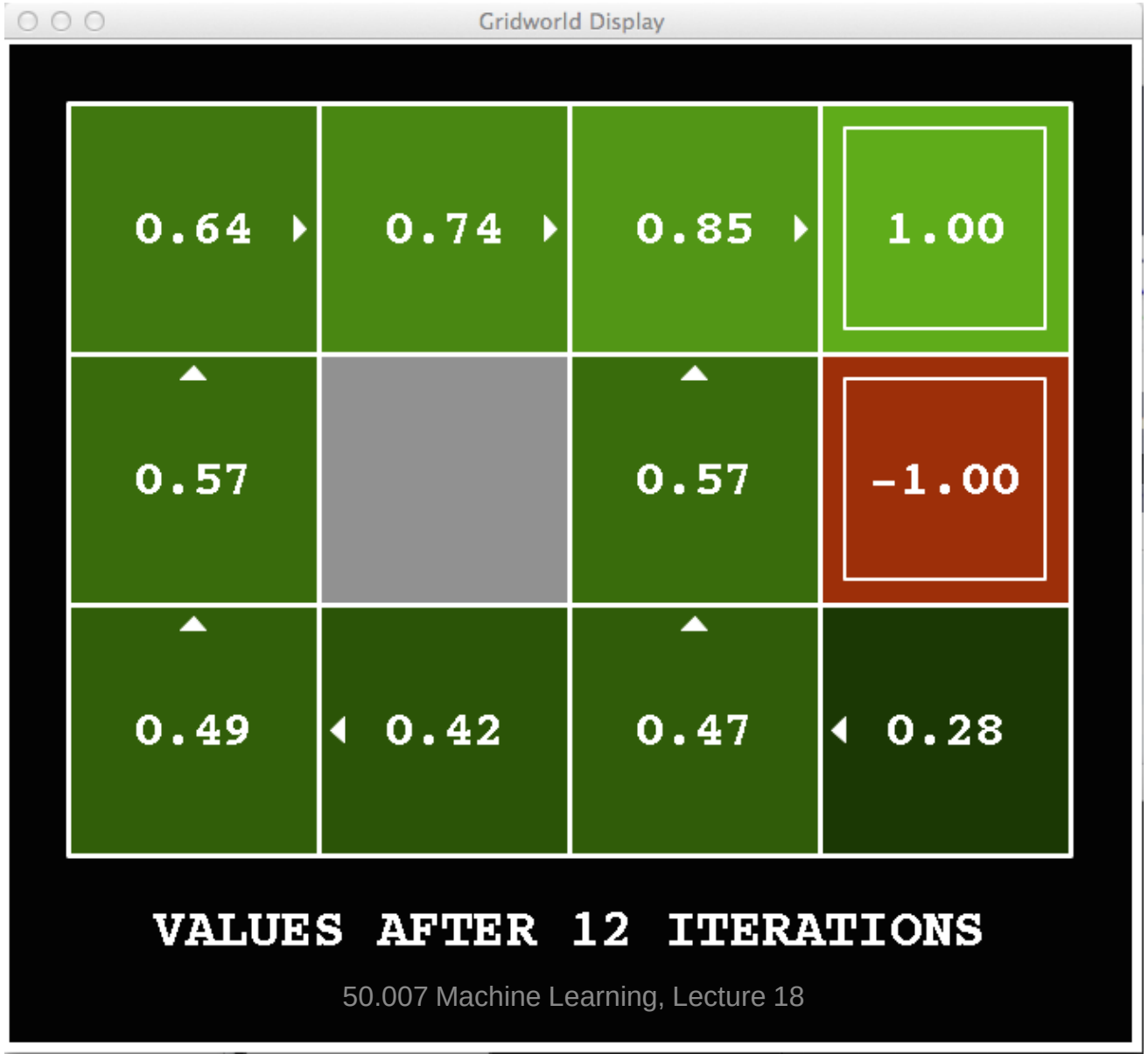


Noise = 0.2
Discount = 0.9
Living reward = 0

i=11

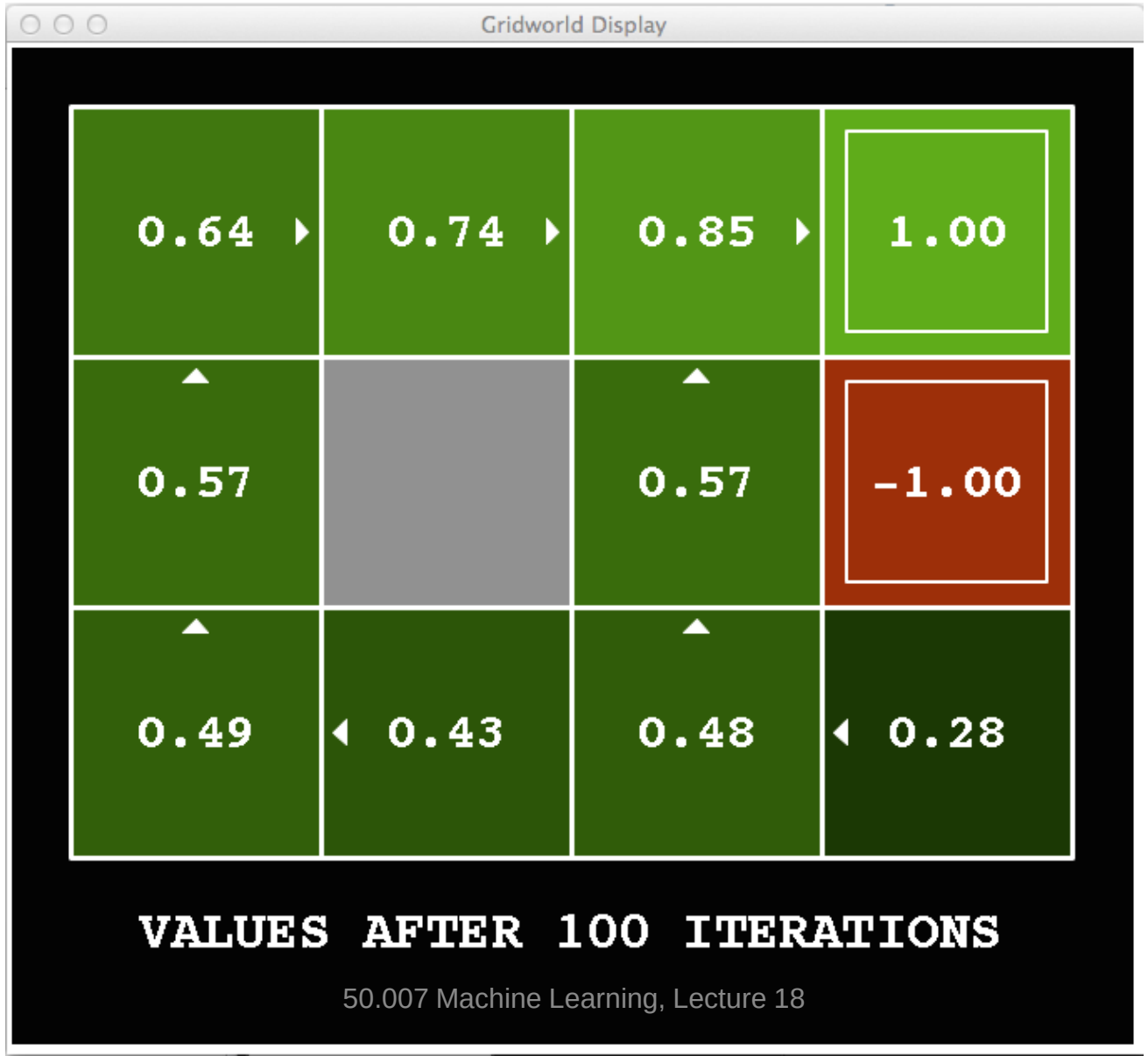
Noise = 0.2
Discount = 0.9
Living reward = 0

i=12



Noise = 0.2
Discount = 0.9
Living reward = 0

i=100



Noise = 0.2
Discount = 0.9
Living reward = 0

Learning the Optimal Policy

Step 1
Run Value Iteration Algorithm

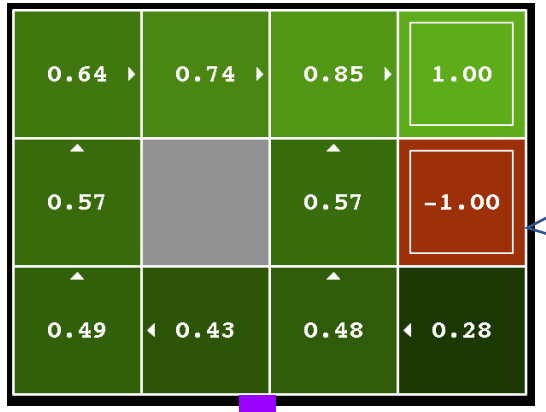
$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Step 2
Calculate the Q values

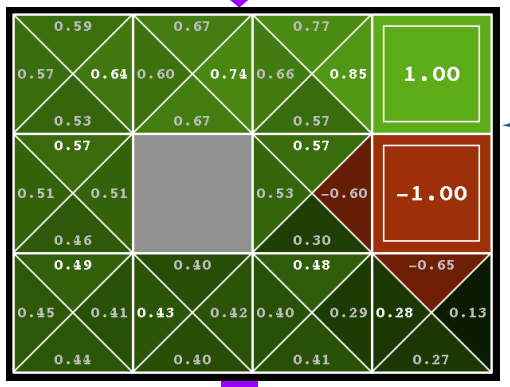
$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Step 3
Find the optimal action for each state

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$



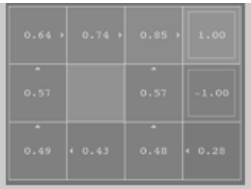
Remember we only kept track of the optimal values and not the actions (i.e. little arrows are not known).



Extra step needed to extract the policy.

Learning the Optimal Policy

Step 1
Run Value Iteration Algorithm

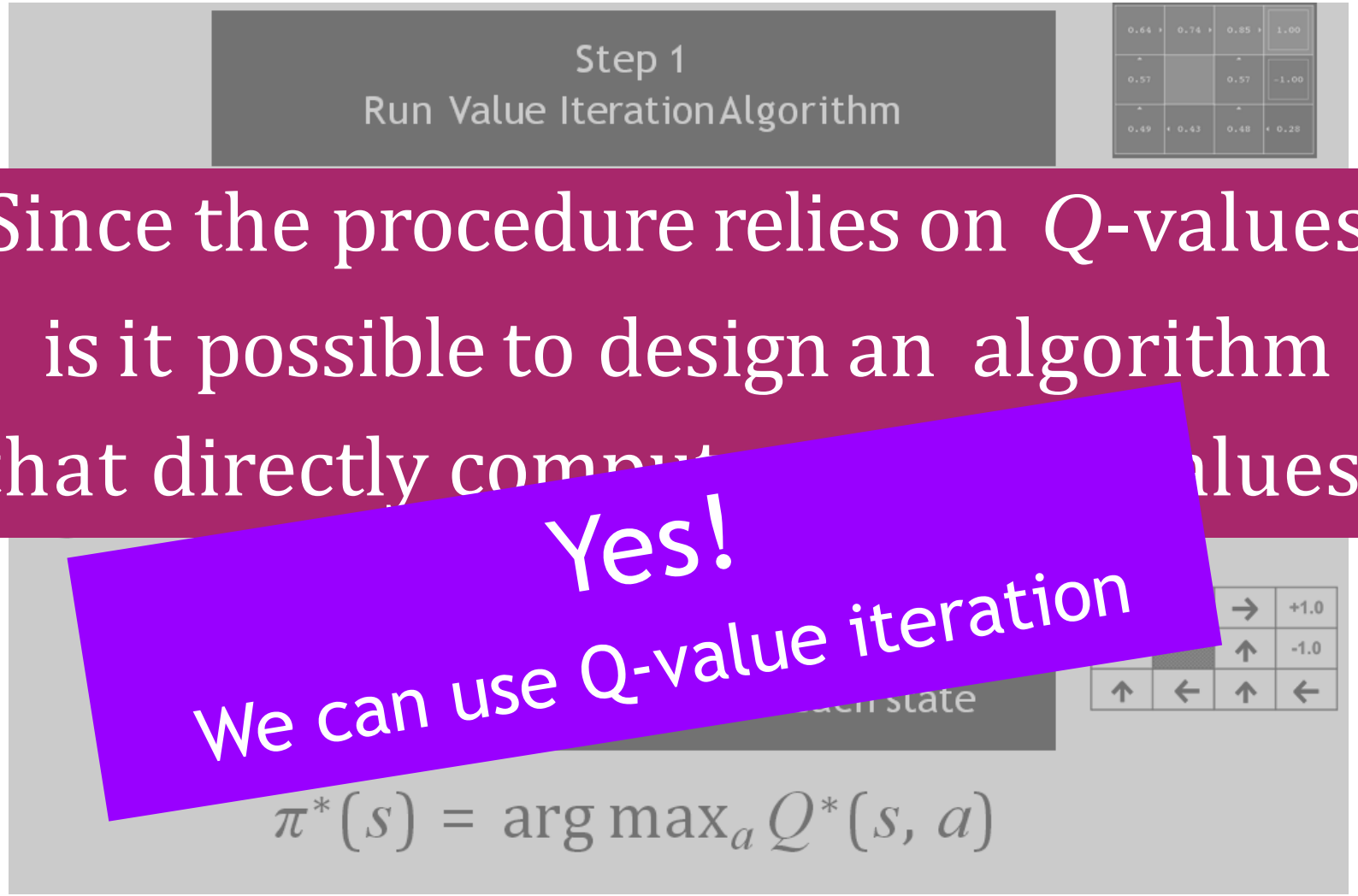


Since the procedure relies on Q-values, is it possible to design an algorithm that directly computes these Q-values?

Step 3
Find the optimal action for each state

$$\pi^*(s) = \arg\max_a Q^*(s, a)$$

Learning the Optimal Policy



Recall: Optimal Value, Q-Value, Policy

$$V^*(s) = \max_a Q^*(s,a) = Q^*(s, \pi^*(s))$$

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Also, known as Bellman Equation

Optimal Value, Q-Value, Policy

$$V^*(s) = \max_a Q^*(s,a) = Q^*(s, \pi^*(s))$$

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Can we have an equation for which both sides involve the Q terms only?

Optimal Value, Q-Value, Policy

$$V^*(s) = \max_a Q^*(s,a) = Q^*(s, \pi^*(s))$$

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Can we have an equation for which both sides involve the Q terms only?

Optimal Value, Q-Value, Policy

$$V^*(s) = \max_a Q^*(s,a) = Q^*(s, \pi^*(s))$$

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

$$V^*(s') = \max_{a'} Q^*(s',a')$$

$$Q^*(s,a) \leftarrow \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')]$$

Q-Value Iteration

1. Start with $Q_0^*(s, a) = 0$ for all $s \in S, a \in A$

2. Given $Q_i^*(s, a)$, calculate the Q-values
for all states (depth $i + 1$) and for all actions a:

$$Q_{i+1}^*(s, a) \leftarrow \sum_{s'} T(s, a, s')[R(s, a, s') + \gamma \max_{a'} Q_i^*(s', a')]$$

3. Repeat the above step until convergence.

Q-Value Iteration

1. Start with $Q_0^*(s,a) = 0$ for all $s \in S, a \in A$

2. Given $Q_i^*(s,a)$, calculate the Q-values for all states (depth $i+1$) and for all actions a:

$$Q_{i+1}^*(s,a) \leftarrow \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma \max_{a'} Q_i^*(s',a')]$$

3. Repeat the above step until convergence.

Again, there is a guarantee it will converge.

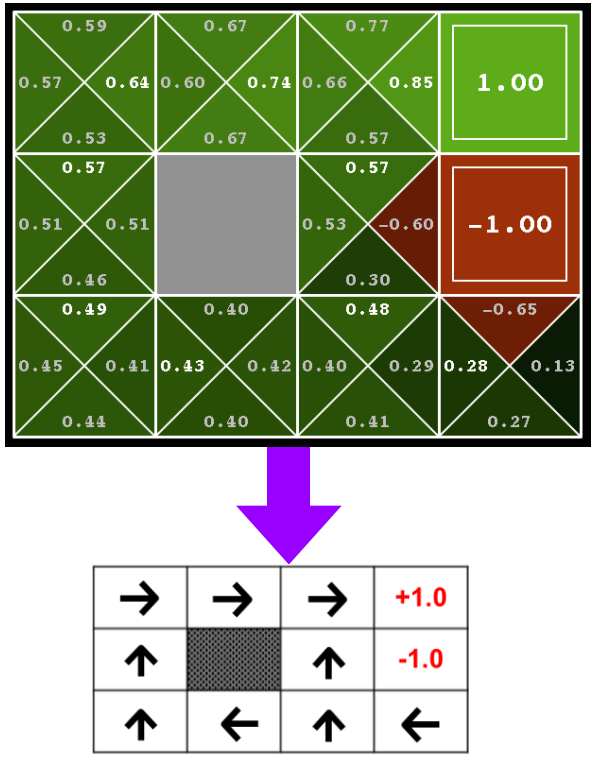
Learning the Optimal Policy

Step1
Run Q-Value Iteration Algorithm

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

Step 2
Find the optimal action for each state

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$

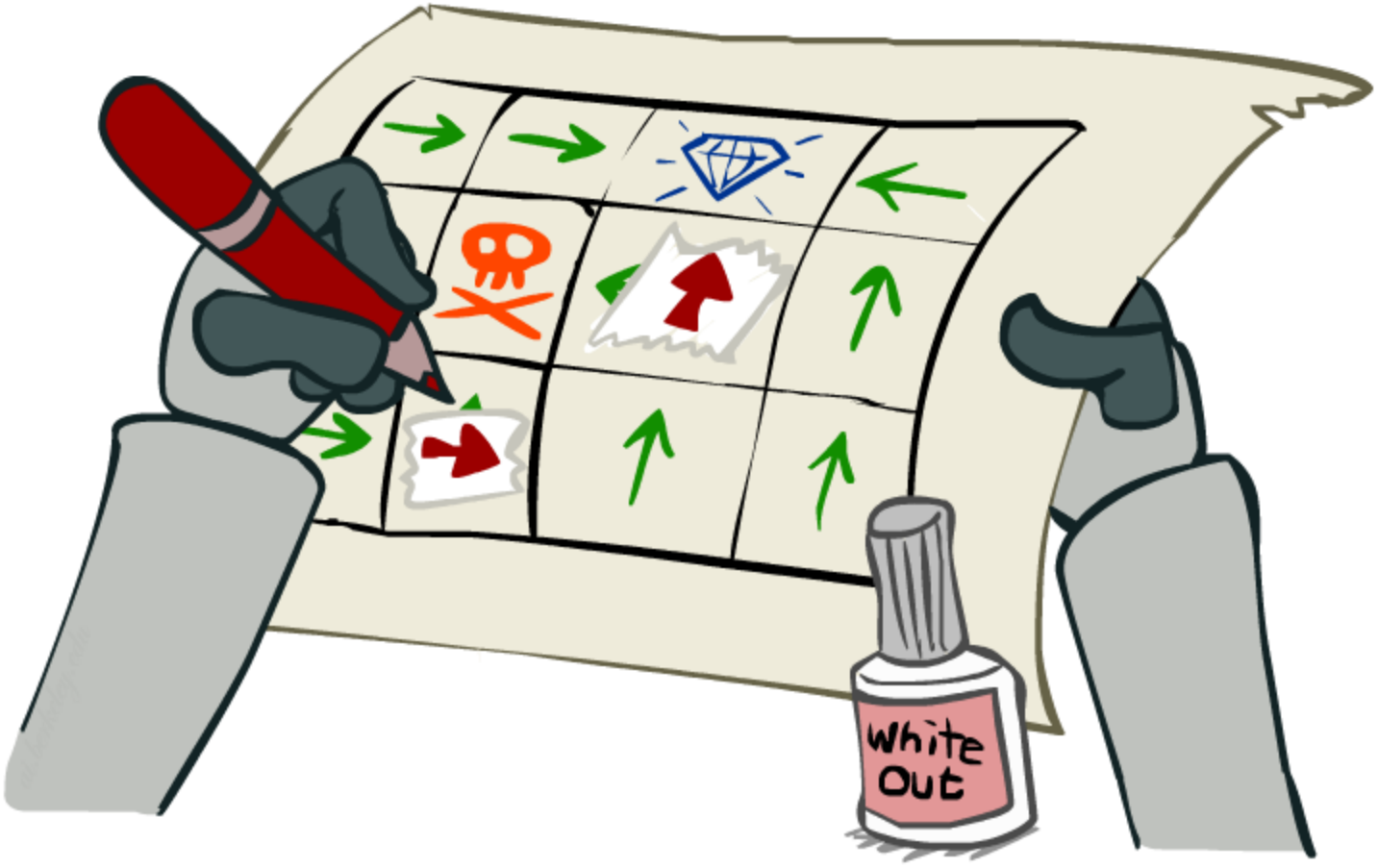


Important lesson: actions are easier to select from Q-values than values!

Problems with Value Iteration

- **Problem 1:** It's slow – $O(S^2A)$ per iteration
- **Problem 2:** The "max" at each state rarely changes
- **Problem 3:** The policy often converges long before the values

Policy Iteration (OPTIONAL)



Policy Iteration

- Alternative approach for optimal values:
  - Step 1: Policy evaluation: calculate utilities for some fixed policy (not optimal utilities!) until convergence
  - Step 2: Policy improvement: update policy using one-step look-ahead with resulting converged (but not optimal!) utilities as future values
  - Repeat steps until policy converges

- This is policy iteration
  - It's still optimal!
  - Can converge (much) faster under some conditions

Policy Iteration

- **Evaluation:** For fixed current policy π, find values with policy evaluation:
  - **Iterate until values converge:**

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') \left[R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s') \right]$$

Alternatively, we can get a linear system of equations to solve. Refer to Eq. 8 from notes for this method of policy evaluation.

- **Improvement:** For fixed values, get a better policy using policy extraction
  - **One-step look-ahead:**

$$\pi_{i+1}(s) = \arg\max_a \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V^{\pi_i}(s') \right]$$

Comparison

- **Both value iteration and policy iteration compute the same thing (all optimal values)**

- **In value iteration:**
  - Every iteration updates both the values and (implicitly) the policy
  - We don’t track the policy, but taking the max over actions implicitly re-computes it

- **In policy iteration:**
  - We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them)
  - After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
  - The new policy will be better (or we’re done)

- **Both are dynamic programs for solving MDPs**

Summary: MDP Algorithms

- So you want to….
  - **Compute optimal values:** use **value iteration or policy iteration**
  - **Compute values for a particular policy**: use **policy evaluation**
  - **Turn your values into a policy**: use **policy extraction** (one-step lookahead)

- These all look the same!
  - **They basically are variations of Bellman updates**
  - They differ only in whether we plug in a fixed policy or max over actions